Measuring the Overhead of Trust Mechanisms in the X-Road Data Exchange Layer

Ruslan Nechshadimov
University of Tartu
Supervisor: Artjom Lind

Draft v0.2, Oktober 14, 2026

Abstract

WIP

1 Introduction

When one organisation acts on data received from another, it depends on something it did not produce and cannot check by itself. Trust is the name for that dependence. In organisational research it is defined as the willingness of a party to be vulnerable to the actions of another party, based on the expectation that the other will perform an action important to the trustor, without being able to monitor or control it [17]. In computing the same idea is phrased as a belief in the competence of an entity to act dependably, securely and reliably within a specified context [18]. In security engineering a *trusted* component is distinguished from a *trustworthy* one: the first is a component whose failure can break the security policy, the second one that will not fail [19]. A well-designed system therefore tries to need as little trust as possible, and to place the rest in a few components that are named explicitly and protected accordingly.

X-Road, the data exchange layer, is built on this principle. Its member organisations do not have to trust each other's systems directly. Their trust rests on a root of trust, a point that is accepted as trusted axiomatically [3]. In X-Road this root is centralised: it is formed by the certification authorities that vouch for the members' keys [3], together with the Central Server, which publishes who the members are and which authorities to trust, and the time-stamping authority [11]. Every message then carries evidence that can be checked against this root. It is signed by the sender's Security Server and verified by the receiver's, written into the message logs on both sides and time-stamped, and it travels over a mutually authenticated Transport Layer Security (TLS) channel, while the status of the certificates is checked through the Online Certificate Status Protocol (OCSP) [11, 1]. Trust in the other party is replaced by verification against the root of trust, and an exchange can later be used as evidence of who sent what and when. The other side of a centralised root is that it becomes a single point of failure [3], which is why alternatives to it are being studied.

Verification is not free. Signing, verification, logging and setting up the channel take computation and bytes, some of them on every message and some in the background. X-Road has been measured before, but end to end, for questions other than the cost of these mechanisms: a study of threshold signatures reports end-to-end round trips of about 32 ms with all components on one laptop and presents them as a lower bound [4]; a capacity study measures the overhead of a Security Server in front of one registry service, where database time dominates [7]; and the paper that introduced batch time-stamping gives an analytical estimate and leaves benchmarking of the implementation to future work [1]. The Nordic Institute for Interoperability Solutions (NIIS), which develops X-Road, does not publish performance data. None of these sources splits the cost of an exchange by mechanism.

Two developments make that missing split more relevant. The first is volume: the design of batch time-stamping was calibrated against about 240 million queries per year [1], while the Estonian instance now reports 3.74 billion [10], so whatever is paid per message is multiplied by a much larger base. The second is a set of proposals that would make the trust mechanisms themselves more expensive. Post-quantum signatures are expected to enlarge certificates, signatures and handshakes [5, 6]; a Dilithium5 signature, for example, takes 4595 bytes against the 256 bytes of RSA-2048 [6]. Decentralised Public Key Infrastructure (PKI) built on verifiable credentials adds a presentation to every request [2], and threshold signatures replace a local key with a distributed protocol [4]. These works describe what will become more expensive, but without a measured baseline for the present system they cannot say by how much.

The proposals above do not change X-Road as a whole but individual mechanisms. Post-quantum algorithms affect signing, verification and the TLS handshake [5], a decentralised PKI changes how certificates are checked [2], and threshold signatures change how messages are signed [4]. The cost of each change can only be judged against the cost of the mechanism it replaces, and since X-Road keeps certificate status checking and time-stamping out of the request path by design [1], it also matters which of these costs are paid on every message. Larger post-quantum signatures and certificates make messages bigger, and the growth in volume means more messages processed at the same time, so the costs have to be known as a function of message size and load. Finally, the most recent figures were obtained with all components on one machine, which their authors present as a lower bound [4], whereas the Security Servers of two organisations normally communicate over a network, and the NIIS report expects post-quantum handshakes to add round trips on it [5]. This leads to the following research questions:

RQ1
: How much do the trust mechanisms of an exchange — the connection between the Security Servers, signing and verification, message logging, time-stamping and certificate status checking — contribute to its latency and to the bytes on the wire, and which of these costs are paid on every message rather than deferred?

RQ2
: How do those contributions change with message size and with the number of concurrent clients?

RQ3
: Does the answer change when the Security Servers run on separate machines instead of one host, and how does it change with a round-trip time added between them?

The contribution is a measurement rather than a new method: a testbed that can be redeployed on one or several hosts, the cost of each mechanism in milliseconds and bytes from three environments, a per-stage breakdown taken from X-Road's own operational monitoring, the effect of a controlled added round-trip time together with a prediction made before that measurement, and the raw data. The study measures X-Road as it runs today, with its default keys: post-quantum schemes are not yet available in X-Road [5, 6], so their effect is discussed rather than measured.

2 Background: what is paid per message

X-Road is a centrally managed distributed data exchange layer. A Central Server distributes the global configuration — the member registry, the trusted Certification Authorities (CAs) and the trusted time-stamping authorities — and each member operates a Security Server that handles key management, signing, secure transport, logging and access control [2, 11].

For this study the mechanisms divide by their position relative to the request path:

- **Synchronous**, paid on every message: the mutually authenticated TLS channel between the Security Servers; signing with the member's signing key; verification of the incoming signature; and writing request and response into the message log, whose bodies are logged in full by default [12]. How much the TLS channel costs per message depends on whether connections are reused [13]. By default they are not: the client-side pool reuse (`pool-enable-connection-reuse`) and the provider-side support for pooled connections (`server-support-c`) are both off, so each message opens a new TCP connection and performs a TLS handshake [12].
- **Asynchronous**, amortised or deferred: certificate status checking, which uses cached OCSP responses refreshed on a fixed interval (20 minutes by default) rather than one query per message; time-stamping, applied in batches over a hash chain of log records (once per minute by default, with a tolerated time-stamping outage of four hours, the parameter `acceptable-timestamp-failure` being 14400 seconds); and the periodic refresh of the global configuration [9, 12, 1]. Because one time stamp covers a whole batch, its cost per message is divided by the size of the batch and falls as the load grows [11, 1].

The split is a documented design decision: caching status responses and deferring time stamps were introduced to keep a per-message PKI operation from becoming both a single point of failure and a latency source [1]. It carries a security price as well, since a cached, nonce-less OCSP response leaves a window during which a revoked certificate is still accepted [2]. The performance side of that trade-off is the subject of this study.

3 Methodology

3.1 Testbed

The testbed runs X-Road 7.8.3 in containers built from the official images: one Central Server, a test CA that also provides the OCSP and time-stamping services, and four Security Servers — a management server, one consumer (*ss1*) and two providers (*ss2*, *ss3*). The measured path is *load generator* → *ss1* → *ss2* → *echo service*; the second provider is used only to check what limits the consumer under load. The echo service answers a REST request with a body of the requested size, filled with random bytes so that the message log cannot compress it, and its own processing time is negligible. The load generator is k6, sending REST `GET` requests in a closed loop with a given number of concurrent clients, so the message size below is the size of the response body.

Every Security Server is limited to 2 CPU cores and 4 GB of memory and the Central Server to 2 GB, in all environments, so that the environments differ in where the containers run rather than in how much hardware each of them receives. The whole testbed, including member and service registration, is set up by scripts from a single composition file and can be redeployed from scratch.

3.2 Environments

Three machines. Three identical laptops (Intel Core i5-8250U, 4 cores and 8 threads, 16 GB of memory) connected by a dedicated wired switch. The first runs the Central Server, the test CA, the management server and the second provider, the second runs the consumer and the load generator, and the third runs the provider and the echo service. The containers have their own addresses on the switch network (Docker `ipvlan`), so the traffic between the Security Servers is not translated by the host and leaves the machine. Clocks are synchronised with chrony to within half a millisecond, which matters when timestamps from two machines are compared.

One machine. The same stack on one of these laptops alone. With the same CPU and the same container limits, the difference to the three-machine environment is the network between the Security Servers and the sharing of one CPU by all components.

Development laptop. The same stack on a newer laptop (Intel Core Ultra 7 265U) under Windows 11, with the containers running in WSL 2. It shows how the absolute numbers depend on the CPU.

Emulated network. In all of the environments above the Security Servers are at most one switch apart, with a round-trip time below a millisecond, whereas the Security Servers of two organisations usually communicate over a longer and less predictable path. From local measurements alone it is not possible to tell whether the results would hold there. A controlled round-trip time is therefore added between the consumer and the provider on the three machines with netem, the network emulator of the Linux kernel [16], which adds a chosen delay to the packets of a given flow. This is not a model of a real wide-area deployment, only of the added round-trip time. The same approach is used to study TLS under different network conditions without Internet-scale infrastructure [15], and the accuracy of the delay netem produces has been examined in [20, 21]. How the delay is applied and verified is described in Section 3.6.

On all machines the CPU runs in its performance power mode, which is checked before every grid and recorded with each run, since the power mode affects the latency.

3.3 Isolating the cost of a mechanism

The trust mechanisms of RQ1 cannot all be switched off one at a time, so each of them is observed in the way that changes as little else as possible (Table 1).

Mechanism	Observed through
The whole X-Road path	*direct* compared with *full*
Inter-server connection policy	*reuse* compared with *full*
Logging of message bodies	*nobody* compared with *full*
Signing and verification	stages of the breakdown (Section 3.4)
Time-stamping	requests to the time-stamping service; *synctsa* as another policy
Certificate status checking	requests to the OCSP responder

Table 1: How the cost of each trust mechanism is observed.

Where a setting removes or changes a mechanism, a configuration is compared with the default one (*full*) and differs from it in that setting only:

- **direct** — plain HTTP from the load generator to the echo service, without X-Road. Its difference to *full* is the whole X-Road path, not any single mechanism.
- **reuse** — connections between the Security Servers are kept open (`pool-enable-connection-reuse`, `server-support-clients-pooled-connections` and `server-connector-max-idle-time`). By default each message opens a new connection [12], so the difference is the effect of the connection policy between the Security Servers: setting up the TCP connection and the TLS session and starting every transfer on a new TCP connection [13]. It does not isolate the TLS handshake.
- **nobody** — message bodies are not written to the message log (`message-body-logging=false`), so the difference is the cost of logging the body.
- **synctsa** — every message is time-stamped synchronously (`timestamp-immediately=true`). This is a different policy rather than the cost of the default batched time-stamping, and is reported as such.

Signing and verification cannot be switched off, so they are observed only in the stages of the breakdown in which they happen, together with the other work done in the same stages, rather than as a difference between configurations.

The two background mechanisms cannot be moved into the request path without changing what is measured. Postponing time stamps by a long interval would violate a documented constraint between the time-stamping interval and the OCSP freshness [12]. Making the OCSP freshness shorter than the interval at which the signer refreshes its cached responses makes every request fail, because the Security Server takes certificate status only from that cache and does not query the responder while it processes a message. For these two mechanisms the cost is therefore observed as the number of requests sent to the time-stamping service and to the OCSP responder during each run and while its messages are being time-stamped.

3.4 What is measured

For each request the load generator records the latency and whether a correct response of the expected size arrived. For each run the bytes exchanged between consumer and provider, counted by `iptables` inside the network namespace of the consumer on the X-Road port, the growth of the message log, and the number of OCSP and time-stamping requests from the CA log are recorded.

X-Road also records four timestamps per request on each Security Server in its operational monitoring database: when the request arrived, when it was forwarded, when the response arrived and when it was returned. Joining the records of the consumer and the provider by the request identifier splits the latency into six consecutive stages: the consumer before sending, the way to the provider (which includes signing and logging on the consumer and setting up the connection), the provider before forwarding (verification and logging), the echo service, the provider returning the response, and the consumer finishing it. The stages add up to the end-to-end time. The breakdown is descriptive: these are timestamps of the measured system itself with a resolution of 1 ms, and the share of requests whose records could be matched on both servers is reported with it. Stages measured within one host are more reliable than the two stages that span hosts, where the residual clock offset enters the result; explanations that involve the network are checked against packet captures (Section 3.6).

3.5 Procedure

Two message sizes are used with one client. Post-quantum signatures and certificates would add a few kilobytes to every message [6], which matters most for small messages, so a 10 KB response represents a small message; a 1 MB response represents a large one, whose body the message log stores in full by default [12]. One client with no other load gives the baseline latency without queueing, and the load is then increased until one of the components runs out of resources [7]: the number of clients grows beyond the two CPU cores available to each Security Server, to 4, 16 and 32, which covers the 10 to 30 concurrent clients used in [7] and reaches the throughput ceiling of the Security Servers. The main grid therefore crosses the five configurations with 10 KB and 1 MB responses at one client and 10 KB at 4, 16 and 32 clients, plus the two-provider case at 4, 16 and 32 clients.

The Security Server runs on the Java virtual machine, whose first requests after a start are slower until the code is compiled, so only the steady state is measured [14]. After every configuration change the Security Servers are restarted and warmed up by a discarded run at four clients, followed by warm-up requests before every measured run. The amount of warm-up is set from a pilot run as the point after which the median latency over consecutive windows of requests stops decreasing. Each run measures 1000 requests, as in [4], and 200 at 1 MB, where every message adds about a megabyte to the message log of each Security Server.

Following [14], every point is measured in several independent runs rather than once. A grid repeats every point five times in random order, and a control run with the default configuration at the start of each repetition and at the end shows the drift of the testbed during the grid. Grids are repeated over several nights and combined. Before the message log is cleaned, the runner

waits until every logged message has its time stamp, so that deferred work is not lost between runs.

Results are reported as the median over repetitions of each run's median, with the interquartile range between repetitions and the number of successful requests.

3.6 Added round-trip time between the Security Servers

A controlled round-trip time is added between the consumer and the provider with netem [16], as done for TLS benchmarking in [15]. Half of the round-trip time is added on each side, in the network namespaces of the two Security Servers and only for packets to the other server, so that all traffic between them is delayed, including the OCSP exchange between Security Servers, while the client, the Central Server and the CA stay on the local network. The delay is fixed, without jitter, since netem reproduces jitter less accurately [21, 20]. The results are reported as the effect of a controlled added round-trip time between the Security Servers, not as a measurement of a real wide-area deployment. A control at zero delay with the same netem rules installed checks that the rules themselves do not change the result.

The added round-trip time (0, 5, 20 and 50 ms) is verified on the measured flow itself: during every run the round-trip time of the consumer's connections to the provider is read from the kernel. Before the grid, a pilot with traffic captures of sequential requests identifies for every setting whether a request opens a new connection, with a full or a resumed TLS handshake, or uses an existing one, and how many times one side waits for the other. From these exchanges a prediction is made of how latency should grow with the round-trip time, and it is compared with the measured slope. Captures of a 1 MB response with and without connection reuse check how the transfer of a large response depends on the state of the TCP connection.

The delay grid crosses *full* and *reuse* with 10 KB and 1 MB responses at one client and 0, 5, 20 and 50 ms, and an extended grid adds 100 KB responses and 4 and 16 clients at 0, 20 and 50 ms.

3.7 Threats to validity

Several properties of the testbed shift the absolute values. The machines are laptops rather than servers, and the development laptop adds the overhead of WSL 2 on Windows. The test CA, OCSP responder and time-stamping service run next to the Security Servers, whereas in production they are external services with their own latency, so the figures for status checking and synchronous time-stamping are optimistic. All intervals of the background mechanisms are left at their defaults, while a production deployment may use longer ones: for the Estonian instance of 2017 a master's thesis documents a global configuration validity of six hours and an OCSP freshness of eight hours [8], so background costs per message would be even lower there than in the testbed.

Other properties change the shares and the spread rather than the absolute values. The echo service has no processing time of its own, which raises the relative share of the trust mechanisms compared with a real service. Under load the spread between repetitions is larger, so for concurrency only throughput ceilings are reported, not differences between configurations. The added round-trip time is emulated: it has no cross traffic and no losses.

Because of these factors the absolute latencies describe this testbed rather than a production deployment. The comparisons are therefore made between configurations on the same machines and under the same conditions, and the results are read primarily as differences between configurations and as their change with message size, load and added round-trip time; absolute values are reported for reference only.

4 Expected outcome

A table mapping each trust mechanism to milliseconds and bytes per message, obtained in several environments, supported by raw data and a testbed that can be redeployed from a single composition file. Beyond the seminar report, the result is a reference point for three lines of work that currently lack one: post-quantum migration [5, 6], decentralised PKI [2] and distributed key management [4], each of which proposes to make trust more robust at a cost that has not been compared with the cost of what X-Road does today.

5 Plan

1. Request virtual machines and ports from the university computing centre (day one; the rest of the work proceeds in parallel).
2. Composition file and testbed, identical across environments; automation of member and service registration.
3. Echo service, load scripts, metric collection, message-log cleanup, and the control run for stability.
4. Environment A, full grid.
5. Environment C (or B, whichever becomes available first), same grid, plus emulated latency.
6. Environment D, reduced grid, if it is kept in scope.
7. Analysis, figures, report.

Steps 2 and 3 are the schedule risk; the waiting time in step 1 is the reason environment B is on the list at all.

TODO: to add in Results and Discussion

Working list for the draft, to be removed before submission.

- CPU power mode: 80 ms in the energy-saving mode against 29 ms in the performance mode, and the per-stage timestamps showing that computation slowed down, not the network.
- Pilot numbers behind the grid: latency at 1, 10, 100 KB and 1 MB (29–33 ms up to 100 KB, doubled at 1 MB), throughput ceiling at about eight clients, warm-up curve ($114 \rightarrow 45$ ms, plateau after 2500–3000 requests) and the 20 % spread between repeated runs.
- The warm-up and other figures differ from the sources: [4] reports means over 1000 requests without describing a warm-up, while thousands of warm-up requests were needed here.
- How many times each grid was repeated in each environment (30 on three machines, 15 on one machine, 20 on the development laptop; delay grids 20 and 15).
- TLS session resumption from a session ticket and the count of round trips per message (3 with a new connection, 1 with reuse, plus the waits found in the captures).
- Short OCSP freshness made every request fail — as an observation supporting that status checking is not in the request path.
- Spread between repetitions under load (25–45 %), the reason only ceilings are reported.
- Absolute values depend on the CPU while the differences between configurations hold across machines (one machine against the development laptop) — for the Discussion.

References

[1] A. Ansper, A. Buldas, M. Freudenthal, J. Willemson. High-Performance Qualified Digital Signatures for X-Road. In *NordSec 2013*, LNCS 8208, pp. 123–138, Springer, 2013.

[2] M. Bakhtina, K. L. Leung, R. Matulevičius, A. Awad, P. Švenda. A Decentralised Public Key Infrastructure for X-Road. In *ARES 2023*, ACM. DOI: 10.1145/3600160.3605092.

[3] M. Bakhtina, R. Matulevičius, A. Awad, P. Kivimäki. On the Shift to Decentralised Identity Management in Distributed Data Exchange Systems. In *SAC '23*, pp. 864–873, ACM, 2023. DOI: 10.1145/3555776.3577678.

[4] M. Bakhtina, J. Kvapil, P. Švenda, R. Matulevičius. Securing Organisational Digital Identity: A Full-Lifecycle Approach to Distributed Key Management. *International Journal of Information Security*, 25:131, 2026. DOI: 10.1007/s10207-026-01291-5.

[5] M. Bakhtina. Migration of X-Road to Post-Quantum Cryptography. NIIS Publication series, Nordic Institute for Interoperability Solutions, 2026. ISBN 978-9916-9853-4-2.

[6] N. Snetkov, J. Vakarjuk. Integrating Post-Quantum Cryptography to UXP. Cybernetica Research Report D-2-499, 2023.

[7] E. Taklai. X-tee teenuste jõudlustestimine Eesti äriregistri näitel (Performance testing of X-Road services on the example of the Estonian Business Register). Diploma thesis, Tallinn University of Technology, 2020.

[8] L. Mironova. X-tee platvormi kriitiliste kesksete teenuste testimine (Testing of the critical central services of the X-Road platform). Master's thesis, Tallinn University of Technology, 2017.

[9] P. Kivimäki. Resisting Failure. NIIS blog, 20 November 2020. `https://www.niis.org/blog/2020/11/20/resisting-failure`.

[10] Estonian Information System Authority (RIA). X-tee Factsheet, Estonia. `https://www.x-tee.ee/factsheets/EE/`. Figures based on data of 1 September 2026; accessed 17 September 2026.

[11] Nordic Institute for Interoperability Solutions. X-Road: Security Server Architecture. `https://docs.x-road.global/Architecture/arc-ss_x-road_security_server_architecture.html`. Accessed 17 September 2026.

[12] Nordic Institute for Interoperability Solutions. X-Road: System Parameters (UG-SYSPAR). `https://docs.x-road.global/Manuals/ug-syspar_x-road_v6_system_parameters.html`. Accessed 17 September 2026.

[13] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, P. Steenkiste. The Cost of the "S" in HTTPS. In *CoNEXT '14*, pp. 133–140, ACM, 2014. DOI: 10.1145/2674005.2674991.

[14] A. Georges, D. Buytaert, L. Eeckhout. Statistically Rigorous Java Performance Evaluation. In *OOPSLA '07*, pp. 57–76, ACM, 2007. DOI: 10.1145/1297027.1297033.

[15] C. Paquin, D. Stebila, G. Tamvada. Benchmarking Post-quantum Cryptography in TLS. In *PQCrypto 2020*, LNCS vol. 12100, pp. 72–91, Springer, 2020. Cryptology ePrint Archive 2019/1447.

[16] S. Hemminger. Network Emulation with NetEm. In *linux.conf.au 2005*, Canberra, 2005.

[17] R. C. Mayer, J. H. Davis, F. D. Schoorman. An Integrative Model of Organizational Trust. *Academy of Management Review*, 20(3):709–734, 1995.

[18] T. Grandison, M. Sloman. A Survey of Trust in Internet Applications. *IEEE Communications Surveys & Tutorials*, 3(4):2–16, 2000.

[19] R. Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.

[20] L. Nussbaum, O. Richard. A Comparative Study of Network Link Emulators. In *Communications and Networking Simulation Symposium (CNS '09)*, 2009.

[21] A. Jurgelionis, J.-P. Laulajainen, M. Hirvonen, A. I. Wang. An Empirical Study of NetEm Network Emulation Functionalities. In *ICCCN 2011*, IEEE, 2011.